DATA STRUCTURE SEMINAR:

TOPIC : ADJACENCY LIST

TEAM 4

Made with GAMMA

TEAM MEMBERS:

- Kashinathan S J
- Deborah Mel Antony
- Arjun Suresh
- Hridhyakalliani
- Surya Kiran S
- Malavika M
- Medha Pradeep

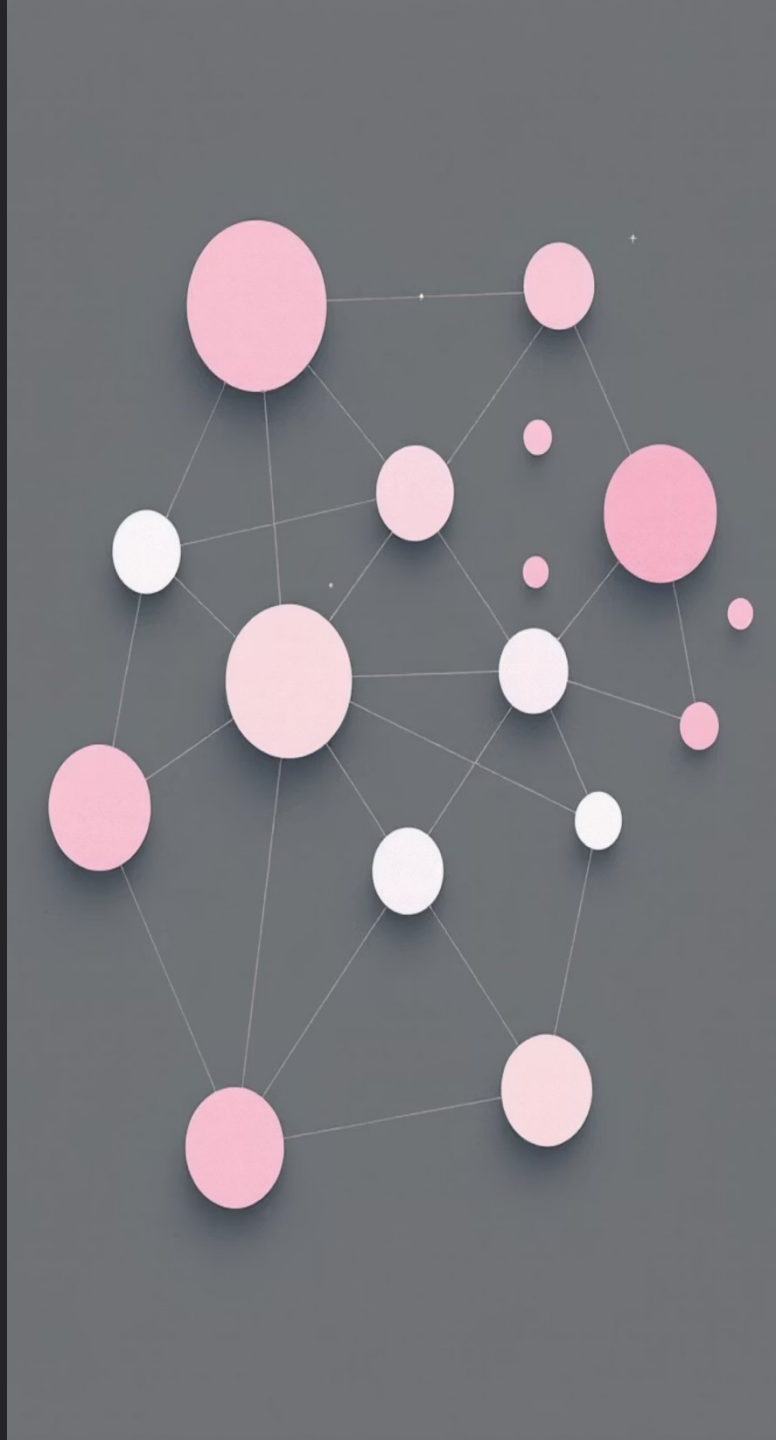
Made with GAMMA

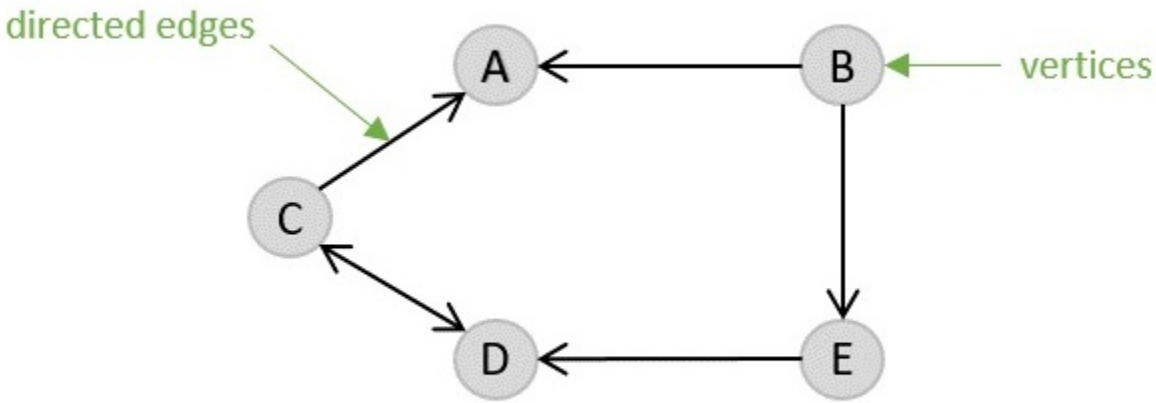
CONTENTS:

- What is a graph
- Adjacency matrix
- Adjacency list
- Directed and Undirected list
- Algorithm of adjacency list
- Operations on adjacency list
- Applications of Adjacency list
- Advantages and Disadvantages

Made with GAMMA

- **A graph is a non-linear data structure consisting of a set of vertices (nodes) and edges that connect pairs of vertices.**
- **Vertices (Nodes):** Points that represent entities or data items.
- **Edges (Links):** Connections between vertices
- **eg: Social Network:** Users are vertices; friendships are edges.
- **Maps/Navigation:** Cities are vertices; roads are edges.
- **Web Links:** Web pages are vertices; hyperlinks are edges.





Graph Data Structure

Made with GAMMA

- In Data Structures and Algorithms (DSA), graphs are represented mainly in two ways: **Adjacency Matrix** and **Adjacency List**.
- **Adjacency matrix:**
- It is a **2D array** used to represent a graph, where each row and column represent vertices.

Made with GAMMA

Adjacency List:

- An adjacency list is a data structure used to represent a graph where each node in the graph stores a list of its neighboring vertices.

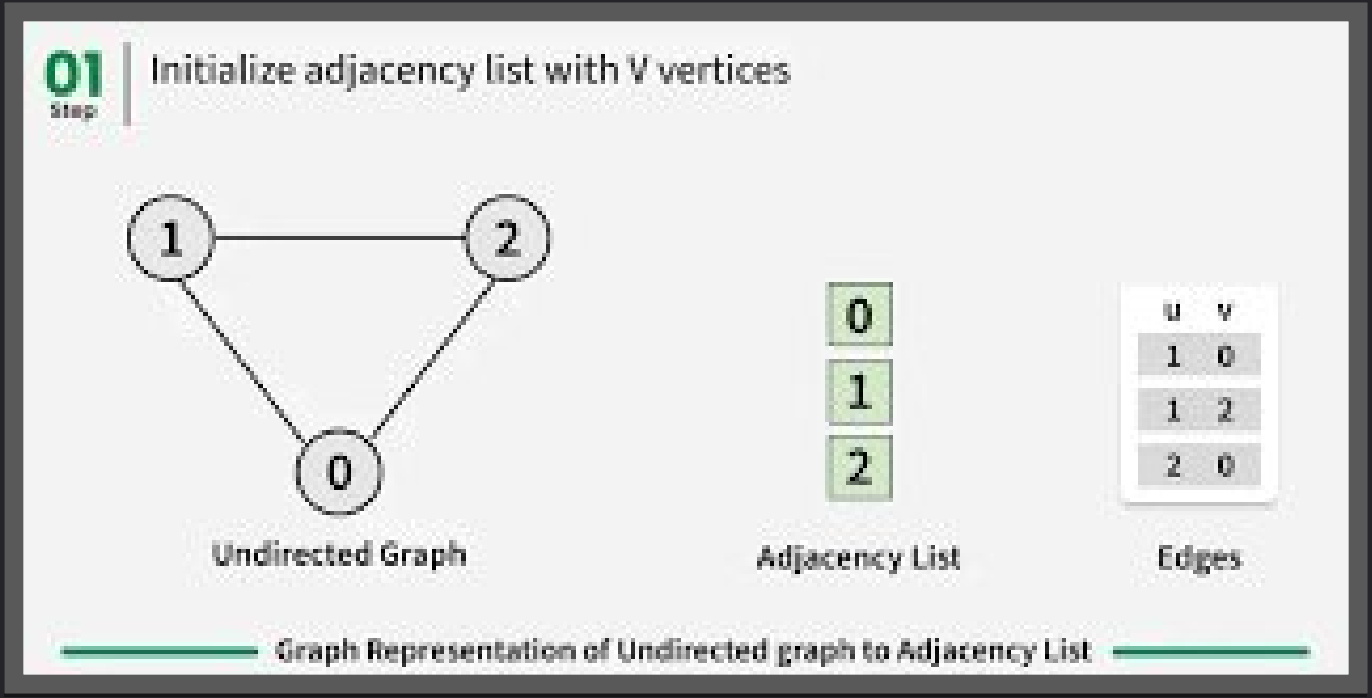
They are of two types:

- For directed graph:
- For undirected graph:

Made with GAMMA

UNDIRECTED GRAPH:

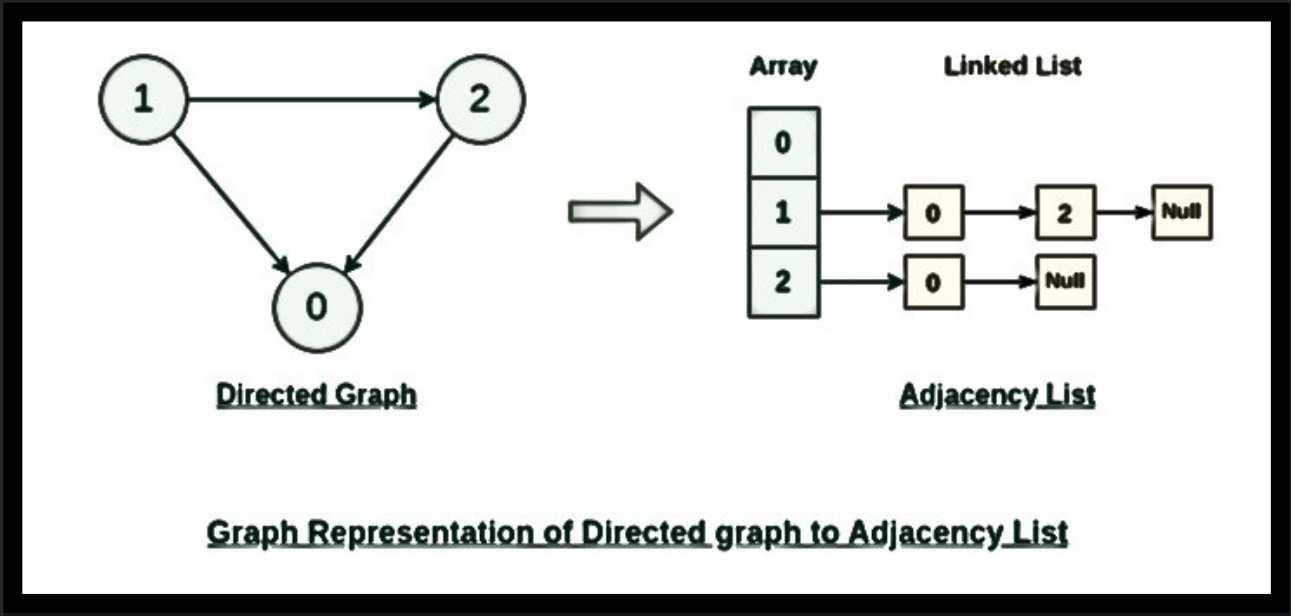
- An **undirected graph** is a graph where edges have **no direction**. That means if there is an edge between A and B, you can go both



Made with GAMMA

DIRECTED GRAPH:

- **Directed graph** is a graph in which the edges have a direction. That is, an edge from vertex A to vertex B does **not** imply edge from B to A.



Made with GAMMA

Adjacency List Algorithm

1. Read the number of vertices V and edges E.
2. Initialize an empty list for each vertex.
3. Loop through each edge (u, v).
4. Add v to the list of u.
5. If undirected, also add u to the list of v.
6. Output or use the adjacency list for graph operations.

Made with GAMMA

Operations on Adjacency List:

Adding an Edge:

- To **connect two vertices** (add an edge):
  - Add the second vertex to the **list of the first vertex**.
  - If it's **undirected**, also add the first vertex to the **list of the second vertex**.
- Example:
- Add edge A -> B

Made with GAMMA

Removing an Edge:

- To **remove a connection** between two vertices:
  - Delete the second vertex from the **first vertex's list**
  - If undirected, delete the first vertex from the second vertex's list too.
  - Example:
  - Remove edge A -> B

Made with GAMMA

Applications of Adjacency List-

***1.Graph Traversal Algorithms*:**

- Used in Breadth-First Search (BFS) and Depth-First Search (DFS) to efficiently explore nodes.
- Enables quick access to adjacent vertices during traversal.

2. Shortest Path Algorithms:

- Powers algorithms like Dijkstra's and Bellman-Ford by storing neighbors and edge weights.
- Especially useful in weighted graphs when implemented with priority queues.

3. Network Routing:

- Models computer networks where nodes are routers and edges are connections.
- Helps simulate and optimize routing protocols.

Made with GAMMA

Implementation of Adjacency List:

- **Breadth First Search (BFS)Type:** Traversal algorithm Purpose: Visits all vertices level by level (like exploring friends of friends).

- **Why adjacency list helps:** Easy to find neighbors of each node.
- **Uses:**Shortest path in unweighted graphs, social networks, etc.

- **Example:**

- Imagine you're on Instagram, and you want to find all users within 3 connections (friends of friends of friends).

- Let's say:You are A Your friends are B and C Their friends are D, E, and FBFS will explore:1. Start from A2. Visit A's friends → B, C3. Then visit friends of B and C → D, E, F👉 It explores level by level, like spreading waves of connections.

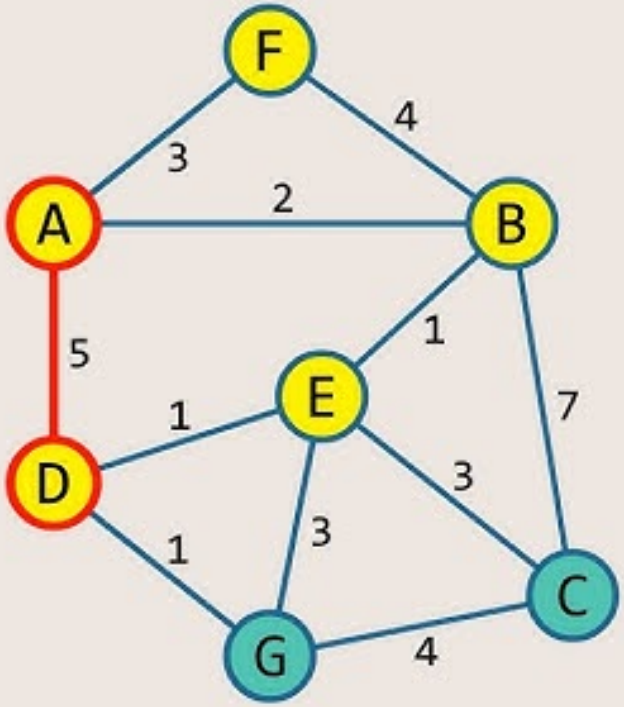
Made with GAMMA

- **2. Depth First Search (DFS)Type:**
- **Traversal algorithmPurpose:** Goes deep into each branch before backtracking.
- **Why adjacency list helps:** Makes recursion or stack-based DFS efficient.
- **Uses:** Detect cycles, connected components, topological sorting.

- **3. Dijkstra's AlgorithmType:**
- **Shortest path algorithm Purpose**: Finds the shortest path from one vertex to all others.
- **Why adjacency list helps:** Reduces time complexity for sparse graphs.
- **Uses:** GPS navigation, network routing.

Made with GAMMA

Dijsktra's Algorithm



Node	Cost	Previous
A	0	
B	2	A
C	6	E
D	4	E
E	3	B
F	3	A
G	6	E

Made with GAMMA

Advantages of Adjacency List:

- ***Space Efficient*** – Uses less memory than an adjacency matrix for sparse graphs because it only stores existing edges.
- ***Easy to Traverse*** – Quickly find all neighbors of a vertex, which is useful in algorithms like DFS and BFS.
- ***Dynamic Graphs*** – Adding or removing vertices and edges is easier compared to an adjacency matrix.
- ***Better for Sparse Graphs*** – If the number of edges is much less than V^2 (V = number of vertices), adjacency list saves a lot of space.

Made with GAMMA

Disadvantages of Adjacency List:

- ***Slower Edge Lookup*** : Finding if an edge exists can take time.
- ***Not Efficient for Dense Graphs*** : More edges make adjacency matrix faster.
- ***Extra Memory for Pointers*** : Linked list nodes need extra memory.
- ***Harder for Certain Operations*** : Some operations like checking all possible edges are slower.

Made with GAMMA


THANK
YOU